tools.py

```python
from __future__ import annotations
from typing import TYPE_CHECKING
from point import Point
import numpy as np

if TYPE_CHECKING:
    from paint import Paint

class Tool:
    def on_mouse_move(self, app: Paint, event):
        pass

    def on_mouse_down(self, app: Paint, event):
        pass

    def on_mouse_drag(self, app: Paint, event):
        pass

    def on_mouse_up(self, app: Paint, event):
        pass

    def on_enter(self, app: Paint):
        pass

class Pencil(Tool):
    def on_mouse_down(self, app: Paint, event):
        self.draw(app, event.x, event.y, app.currentColor)

    def on_mouse_drag(self, app: Paint, event):
        app.line(app.lastPos, Point(event.x, event.y))
        app.lastPos = Point(event.x, event.y)

    def on_mouse_up(self, app: Paint, event):
        app.currentStroke.append((event.x, event.y))
        app.currentStroke = []

    def draw(self, app: Paint, x, y, color):
        if app.onCanvas(x, y):
            app.buffer[y, x] = color

class Bucket(Tool):
    def on_mouse_down(self, app: Paint, event):
        app.fill(Point(event.x, event.y))

class Eraser(Tool):
    def on_mouse_down(self, app: Paint, event):
        self.erace(app, event.x, event.y)
```

```
48
49      def on_mouse_drag(self, app: Paint, event):
50          num_steps = max(np.abs(np.array((app.lastPos.x - event.x, app.lastPos.y -
    event.y)))) + 1
51          points = np.round(np.linspace((app.lastPos.x, app.lastPos.y), (event.x,
    event.y), num_steps, retstep = False)).astype(int)
52
53          app.lastPos = Point(event.x, event.y)
54
55          for point in points:
56              self.erace(app, point[0], point[1])
57              app.currentStroke.append((int(point[0]), int(point[1])))
58
59      def on_mouse_up(self, app: Paint, event):
60          app.currentStroke.append((event.x, event.y))
61          app.currentStroke = []
62
63      def erace(self, app: Paint, x, y):
64          if app.onCanvas(x, y):
65              app.buffer[y, x] = (255, 255, 255)
66
67
68  class Eyedropper(Tool):
69      def on_mouse_down(self, app: Paint, event):
70          app.colorBar.setPrimaryColor(
71              '#{:02x}{:02x}{:02x}'.format(*
72              tuple(app.buffer[event.y, event.x]
73          )))
74
75  class Rectangle(Tool):
76      def __init__(self):
77          self.visual = 0
78
79      def on_mouse_down(self, app: Paint, event):
80          self.start = Point(event.x, event.y)
81
82      def on_mouse_drag(self, app: Paint, event):
83          self.stop = Point(event.x, event.y)
84          self.create_visual(app)
85
86      def on_mouse_up(self, app: Paint, event):
87          self.stop = Point(event.x, event.y)
88          app.rect(self.start, self.stop)
89          app.render()
90
91      def create_visual(self, app: Paint):
92          x1, x2 = sorted([self.start.x, self.stop.x])
93          y1, y2 = sorted([self.start.y, self.stop.y])
94
```

```
        app.rect(Point(x1, y1), Point(x2, y2), temp = True)
        app.render()

class Line(Tool):
    def on_mouse_down(self, app: Paint, event):
        self.last = Point(event.x, event.y)

    def on_mouse_up(self, app: Paint, event):
        app.line(self.last, Point(event.x, event.y))
        app.render()

    def on_mouse_drag(self, app: Paint, event):
        app.line(self.last, Point(event.x, event.y), temp = True)
        app.render()

class CustomShape(Tool):
    def __init__(self):
        self.points: list[Point] = []

    def on_mouse_up(self, app: Paint, event):
        if self.points:
            app.line(self.points[-1], Point(event.x, event.y))
        self.points.append(Point(event.x, event.y))
        app.render()

    def on_mouse_move(self, app: Paint, event):
        if self.points:
            self.create_visual(app, Point(event.x, event.y))

    def on_enter(self, app: Paint):
        app.line(self.points[-1], self.points[0])
        self.points = []
        app.clearTempBuffer()
        app.render()

    def create_visual(self, app: Paint, mousePos: Point):
        app.line(self.points[-1], mousePos, temp = True)
        app.render()

class Ellipse(Tool):
    def __init__(self):
        self.visual = 0

    def on_mouse_down(self, app: Paint, event):
        self.start = Point(event.x, event.y)

    def on_mouse_drag(self, app: Paint, event):
        self.stop = Point(event.x, event.y)
        self.create_visual(app)
```

```
144
145     def on_mouse_up(self, app: Paint, event):
146         self.stop = Point(event.x, event.y)
147         app.ellipse(self.start, self.stop)
148
149     def create_visual(self, app: Paint):
150         app.ellipse(self.start, self.stop, temp = True)
```